Convolutional Neural Networks

Emre Ozfatura[1]

[1]Intelligent Systems Lab.
Sabanci University, Turkey

2 March 2026

Efficient CNNs

- CNNs are computationally less costly compared to fully connected layers.
- However, in deep architectures, as the number of channel increases, we need larger filters.
- For example, a filter with parameters $C_{in} = 256, C_{out} = 512, kh = 3, kw = 3$ have around 10^6 number of weights.
- Here, we will discus how to reduce the computational complexity of the CNNs.

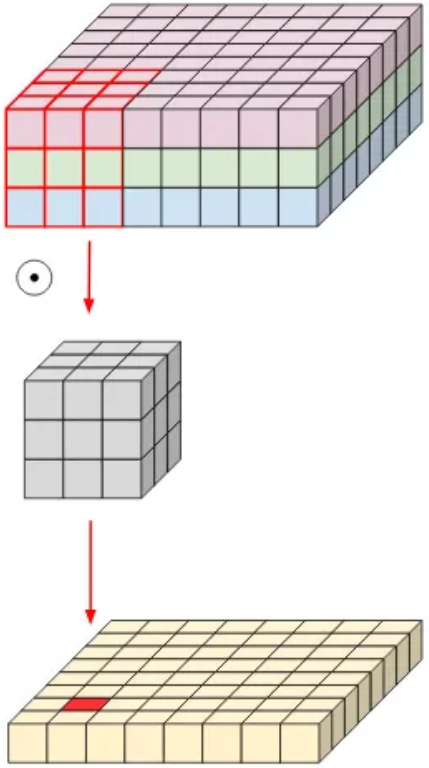


Figure 1: Illustration of standard convolution

Notice: For a single filter its 3D dot product followed by summation:

$$\mathbf{Y}[i,j] = \sum_{c=0}^{C_{in}-1} \sum_{m=0}^{kh-1} \sum_{n=0}^{kw-1} \mathbf{H}[c,m,n]\,\mathbf{X}[c,i,j]. \tag{1}$$

Depth-wise convolution:

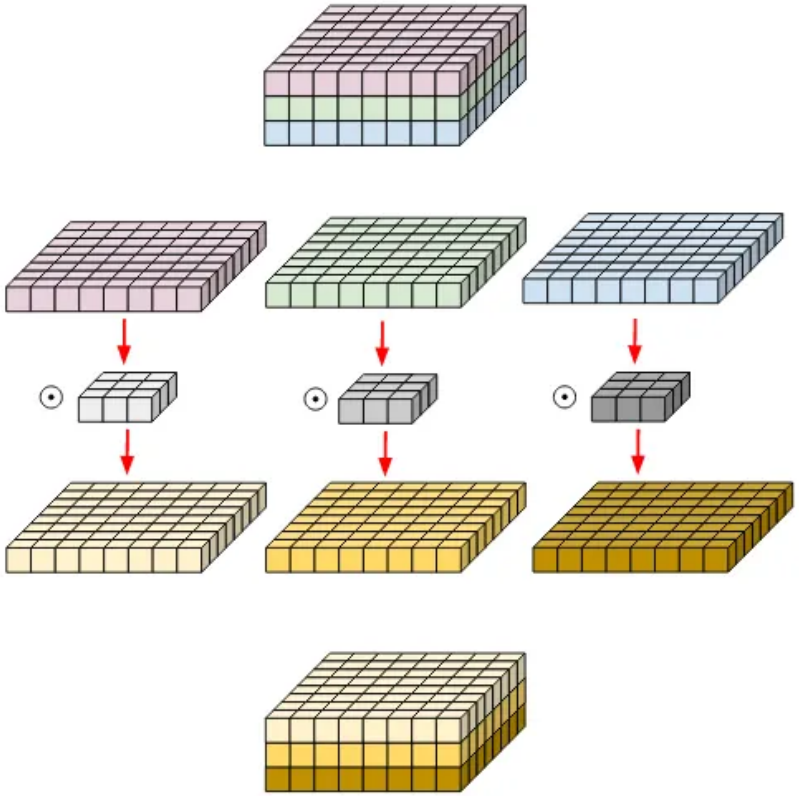
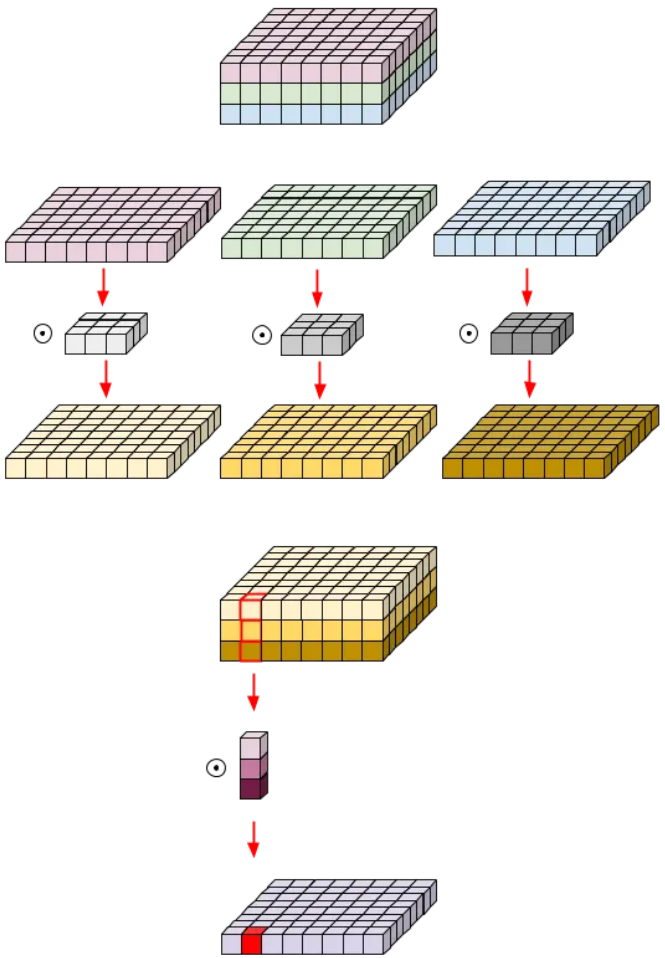


Figure 2: Illustration of depth-wise convolution

Notice: Each layer of the filter is used for a separate channel, then, the results are concatenated in the channel dimension.

Depthwise separable convolution (DwConv)



Notice: The combination of depthwise convolution and point-wise convolution (Conv 1×1) is called depthwise separable convolution.

MobileNet

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
$5\times$ Conv dw / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

Figure 3: Overall architecture.

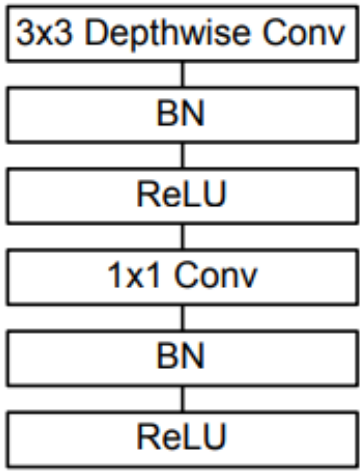


Figure 4: Underlying depth-wise separable convolution block.

Bottleneck to inverted bottleneck

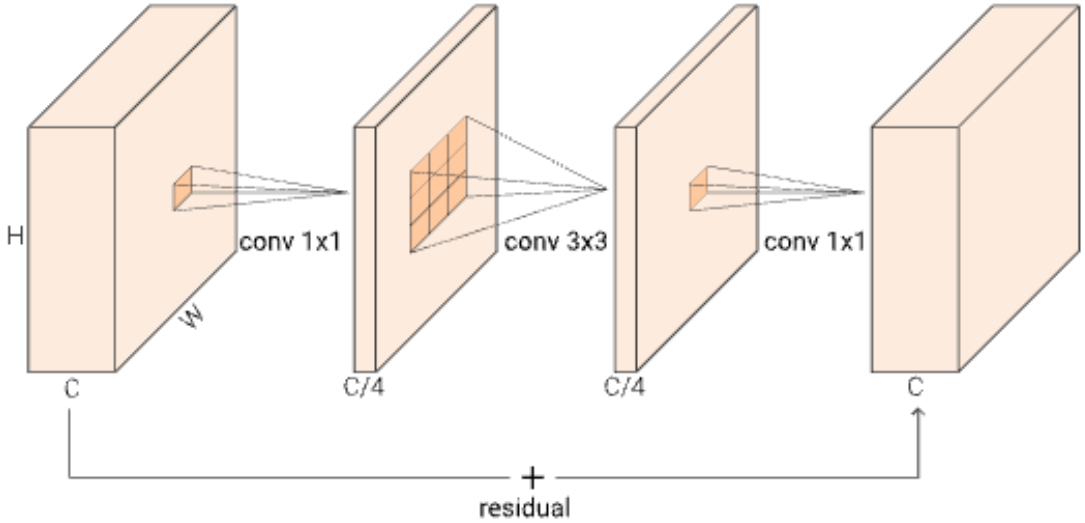


Figure 5: Illustration of bottleneck block.

Recall: We reduce the depth using point-wise convolution (Conv 1×1) to reduce the computation.

Bottleneck to inverted bottleneck

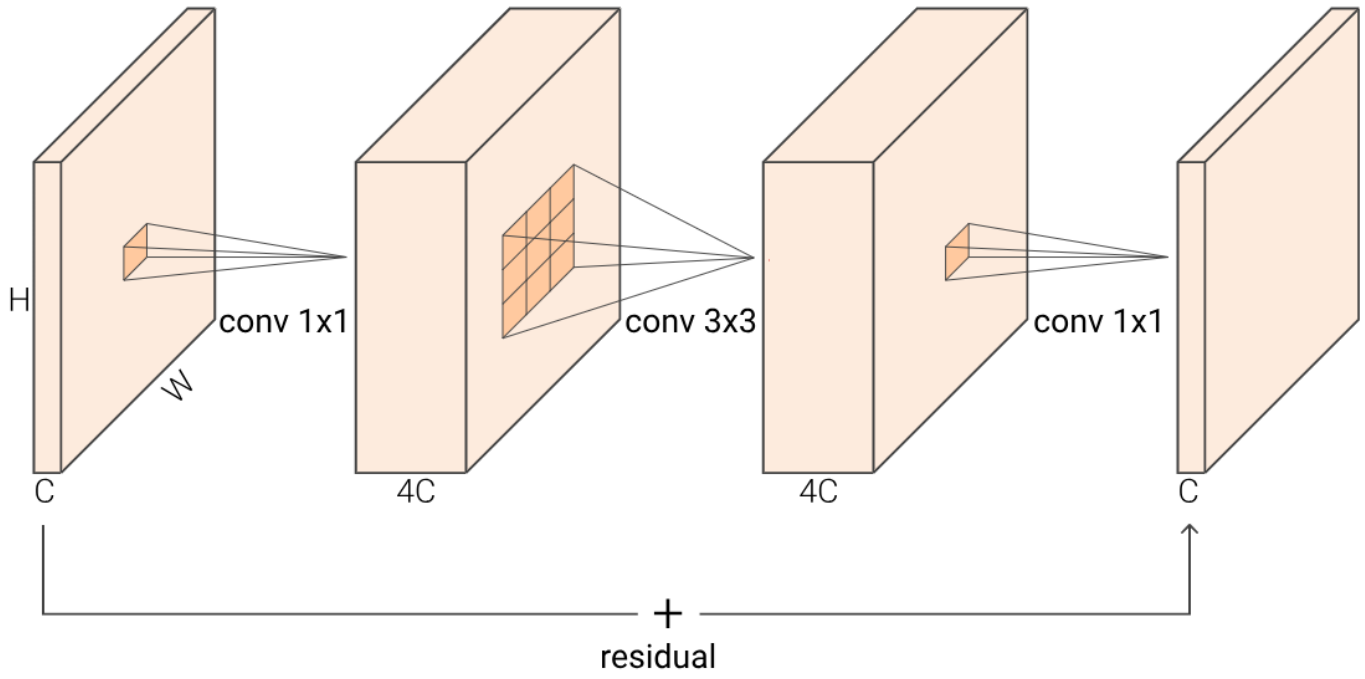


Figure 6: Illustration of inverted bottleneck block.

Notice: Now, we increase the number of channel using point-wise convolution (Conv 1×1). **WHY!!!**

MobileNet Conv (MbConv)

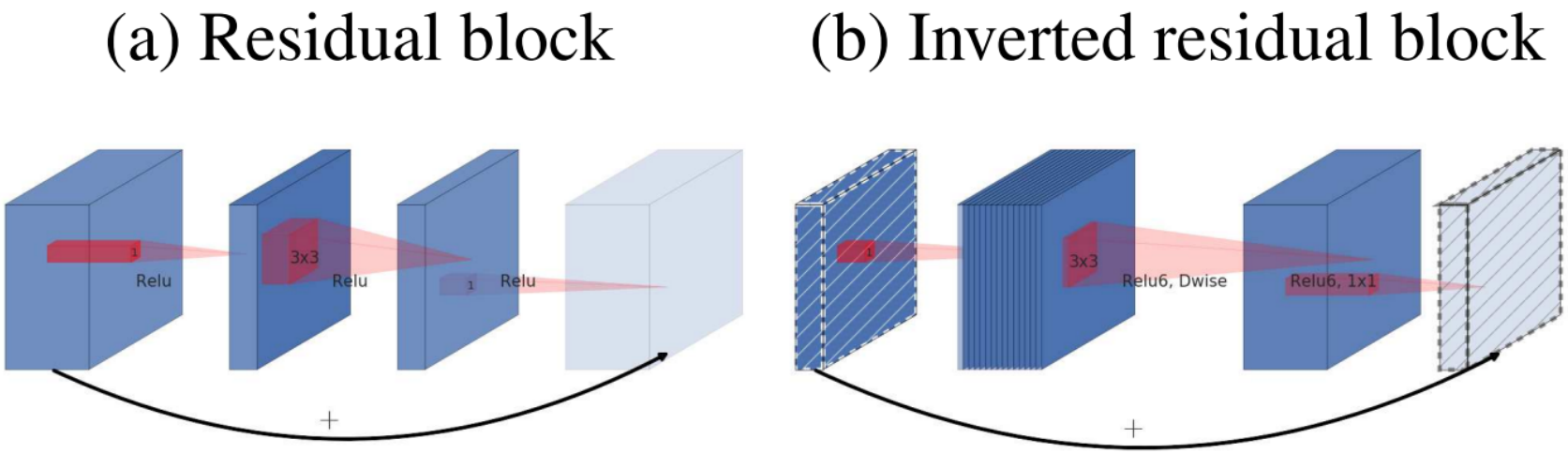


Figure 7: Inverted bottleneck/residual in MobileNet-V2

Notice: We increase the depth, however, use DwConv instead of Conv which reduces the complexity. **Note:** This inverted block also referred as MbConv block in the literature.

MobileNet-V2

- Use inverted bottleneck with depth-wise convolution as main block
- Use ReLU6 $\mathrm{ReLU6}(x) = \min\big(\max(0, x), 6\big)$, for numerical stability.

Input	Operator	t	c	n	s
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$14^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1x1	-	1280	1	1
$7^2 \times 1280$	avgpool 7x7	-	-	1	-
$1 \times 1 \times 1280$	conv2d 1x1	-	k	-	

Figure 8: Overall architecture.

Input	Operator	Output
$h \times w \times k$	1x1 conv2d , ReLU6	$h \times w \times (tk)$
$h \times w \times tk$	3x3 dwise s=s, ReLU6	$\frac{h}{s} \times \frac{w}{s} \times (tk)$
$\frac{h}{s} \times \frac{w}{s} \times tk$	linear 1x1 conv2d	$\frac{h}{s} \times \frac{w}{s} \times k'$

Figure 9: Parameters of MbConv block.

Squeeze-and-Excitation (SE) Block

- Let the input feature map be

$$\mathbf{X} \in \mathbb{R}^{C \times H \times W}.$$

- Global Average Pooling is applied to generate channel descriptors:

$$z_c = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} \mathbf{X}_c(i,j),$$

which produces

$$\mathbf{z} \in \mathbb{R}^{C}.$$

Squeeze-and-Excitation (SE) Block

- The channel descriptor is passed through a bottleneck MLP:

$$\mathbf{s} = \sigma\left(\mathbf{W}_2\, \delta(\mathbf{W}_1 \mathbf{z})\right),$$

  where

$$\mathbf{W}_1 \in \mathbb{R}^{\frac{C}{r} \times C}, \quad \mathbf{W}_2 \in \mathbb{R}^{C \times \frac{C}{r}},$$

$$\delta(\cdot) = \text{ReLU}, \quad \sigma(\cdot) = \text{Sigmoid},$$

  and r is the reduction ratio.

- The original feature map is re-calibrated channel-wise:

$$\tilde{\mathbf{X}}_c = s_c \cdot \mathbf{X}_c.$$

Squeeze-and-Excitation (SE) Block

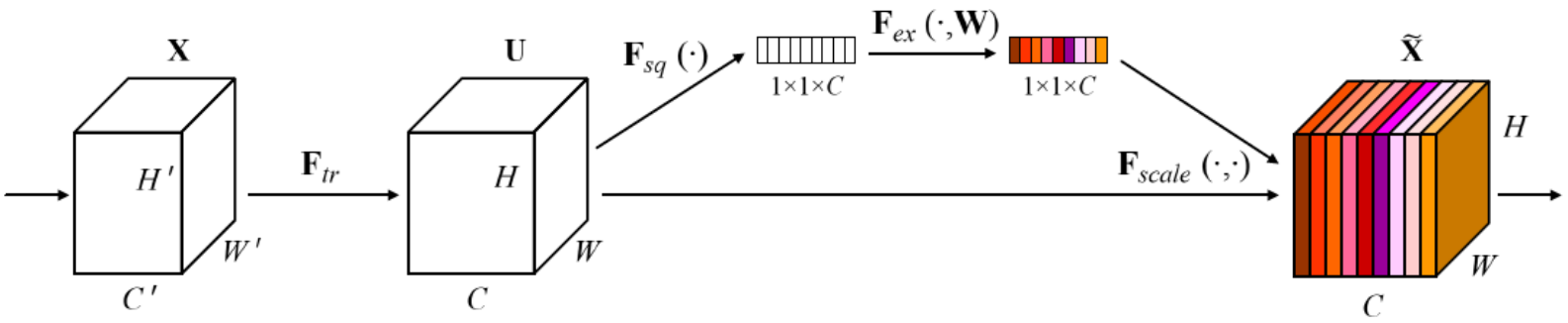


Figure 10: Illustration of Squeeze-and-Excitation (SE) Block

Squeeze-and-Excitation (SE) Block

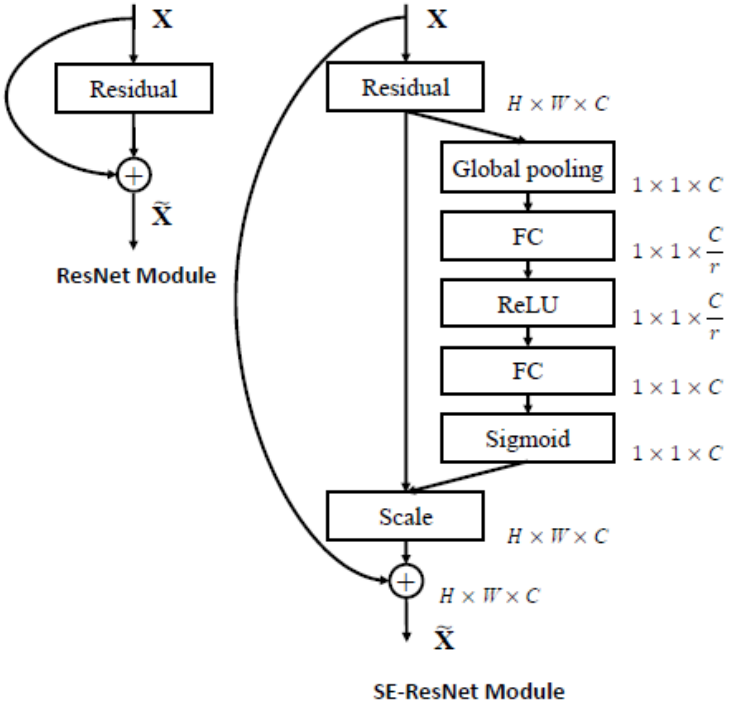


Figure 11: Illustration of Squeeze-and-Excitation (SE) Block in ResNet

Squeeze-and-Excitation (SE) Block

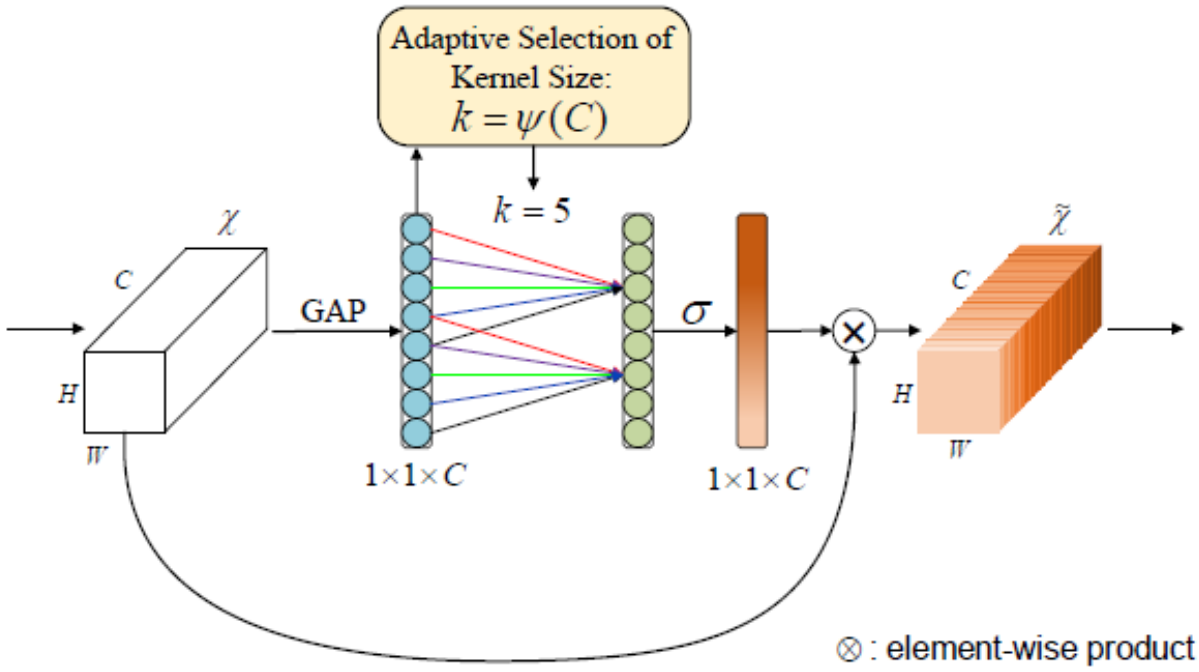


Figure 12: Illustration of efficient implementation of Squeeze-and-Excitation (SE) Block

References

- https://visionbook.mit.edu/convolutional_neural_nets.html
- https://github.com/FrancescoSaverioZuppichini/BottleNeck-InvertedResidual-FusedMBConv-in-PyTorch
- https://medium.com/@zurister/depth-wise-convolution-and-depth-wise-separable-convol
- https://www.youtube.com/watch?v=vVaRhZXovbw
- https://tvm.d2l.ai/chapter_common_operators/depthwise_conv.html

References: papers

- MobileNet - Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., ... & Adam, H. (2017). Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). Mobilenetv2: Inverted residuals and linear bottlenecks. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 4510-4520).

References: papers

- ....